

# TokenVault

## Digital Licensing Clearance Token Bank

Project Documentation  
DSA-I Using C++ — Project VIVA Submission

|                |                                                                        |
|----------------|------------------------------------------------------------------------|
| Student Name   | Tanish Ramesh Suvarna                                                  |
| Enrollment No. | 150096725082                                                           |
| Course         | B.Tech CSE (1st Year)                                                  |
| Cohort         | Larry Page                                                             |
| Semester       | Sem 2                                                                  |
| Subject        | DSA-I Using C++                                                        |
| Instructor     | Aarti Shalindrasingh Pardeshi (Head, B.Tech CSE, School of FutureTech) |
| University     | ITM Skills University                                                  |
| Date           | 16 June 2026                                                           |

### 3.3 Project Title

**TokenVault — Digital Licensing Clearance Token Bank.** A console-based C++17 system that manages a digital token platform end to end, demonstrating eight core Data Structures and Algorithms, each selected to match a specific real-world access pattern and justified against the alternatives.

### 3.4 Problem Statement

TokenVault is a digital asset platform that manages token balances for millions of users, verifies cryptographic keys for every transaction, and facilitates trades across decentralized exchanges. In its current state it suffers from several problems: allocating and tracking balance accounts is slow as it scales; a mistakenly revoked minting authority cannot be traced or reversed; transfer requests pile up out of order; validating a public key requires scanning the entire registry; there is no map of how liquidity pools connect, making safe multi-hop conversion routing impossible; and block storage is wasted for lack of intelligent metadata packing.

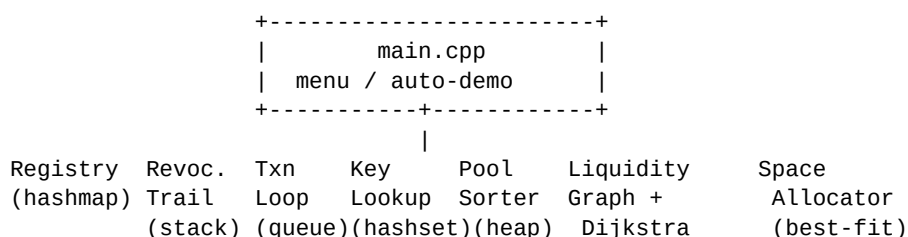
The system must provide fast account management, reliable authority-revocation rollback, strict in-order transaction processing, instant key validation, ranking of mint pools by speed, a liquidity-pool connection map, the lowest-risk multi-step conversion route, and best-fit block-storage allocation.

### 3.5 Objectives

1. Manage token-balance accounts with constant-time create / update / lookup.
2. Maintain an undoable trail of authority grants and revokes for instant rollback.
3. Process transactions strictly in submission order to keep balances consistent.
4. Validate public keys instantly without scanning the full registry.
5. Rank mint pools by velocity and surface the fastest for routing.
6. Model liquidity pools and swap pairs as a graph.
7. Compute the lowest-risk/fee conversion route across multiple hops.
8. Allocate block storage using best-fit to minimize wasted space.
9. Justify every data-structure choice and relate it to Uniswap / Coinbase.

### 3.6 Design / Architecture

The system is a single console application composed of eight independent modules, one per feature. Each module is a self-contained class with its own header (interface plus a justification comment) and implementation file. A central driver, `main.cpp`, holds the shared system state, wires the modules together, and exposes both an interactive menu and a one-shot auto-demo. Modules are decoupled and interact only through the driver — for example, the Transaction Loop calls into the Registry to settle balances.



## 3.7 Algorithm Description

Five features use Standard Library structures applied to the right access pattern; three are implemented from scratch because the algorithmic logic is the core of the exercise. The custom ones are described in detail below.

### Connection Map (graph, adjacency list).

Each liquidity pool is a node; each swap pair is an edge whose weight is risk + fee. Pools are stored in a name→index map; each index holds a vector of outgoing edges (neighbour index, weight). Because the graph is sparse, the adjacency list uses  $O(V + E)$  space versus  $O(V^2)$  for a matrix.

### Route Path (Dijkstra's shortest path).

To find the lowest-cost conversion route, Dijkstra's algorithm runs over the graph with a min-heap keyed by cumulative cost. Starting from the source with distance 0, it repeatedly extracts the cheapest unvisited node and relaxes its edges, recording each node's predecessor so the path can be reconstructed at the end. All weights (risk + fee) are non-negative, which is exactly the precondition Dijkstra requires. Complexity is  $O((V + E) \log V)$ .

```
dist[source] = 0; push (0, source) to min-heap
while heap not empty:
    (cost, u) = heap.pop()           # cheapest so far
    if cost > dist[u]: continue     # stale entry
    for each edge (u -> v, w):
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            prev[v] = u
            push (dist[v], v)
reconstruct path from dst back through prev[]
```

### Space Allocator (best-fit bin packing).

When new metadata of a given size arrives, the allocator scans all blocks and places it in the one whose free space is the smallest that still fits — the tightest fit — minimising leftover waste. First-Fit would stop at the first block that fits (faster, but more waste); Worst-Fit picks the largest gap (maximising leftover, the opposite goal).

```
bestLeftover = infinity; bestBlock = none
for each block:
    free = capacity - used
    if free >= size and (free - size) < bestLeftover:
        bestLeftover = free - size
        bestBlock = block
place size into bestBlock (if found)
```

## 3.8 Complexity Analysis

| Feature               | Structure / Algorithm      | Time                               | Space      |
|-----------------------|----------------------------|------------------------------------|------------|
| Registration Registry | Hash table (unordered_map) | $O(1)$ avg                         | $O(n)$     |
| Revocation Trail      | Stack                      | $O(1)$                             | $O(a)$     |
| Transaction Loop      | Queue                      | $O(1)$                             | $O(t)$     |
| Key Lookup            | Hash set (unordered_set)   | $O(1)$ avg                         | $O(k)$     |
| Pool Sorter           | Max-heap (priority_queue)  | update $O(\log p)$ , peek $O(1)^*$ | $O(p)$     |
| Connection Map        | Graph (adjacency list)     | add $O(1)$ amortized               | $O(V + E)$ |
| Route Path            | Dijkstra + min-heap        | $O((V + E) \log V)$                | $O(V)$     |
| Space Allocator       | Best-fit bin packing       | $O(b)$                             | $O(b)$     |

\*Peek may discard stale heap entries, but each entry is discarded at most once, so the cost is amortized  $O(1)$  across the run. (n = accounts, a = actions, t = pending txns, k = keys, p = pools, b = blocks, V = nodes, E = edges.)

## 3.9 Screenshots of Execution

Screenshots of the program running are maintained in the repository's screenshots/ folder and should be inserted here in the final submission: the interactive menu on launch, the full auto-demo output, the grant→revoke→undo sequence, and a route-path query. A captured text transcript of the auto-demo is shown in section 3.10.

## 3.10 Results and Findings

Captured auto-demo output (all eight features):

1. Registration Registry : alice balance 1000, key PK\_ALICE
2. Revocation Trail : Undid REVOKE of minter\_Y (authority restored)
3. Transaction Loop : tx#1, tx#2, tx#3 settled in submission order
4. Key Lookup : PK\_ALICE valid? true | PK\_FAKE valid? false
5. Pool Sorter : fastest poolB (340) -> after update poolA (999)
- 6+7. Route Path : USDC -> ETH -> DAI -> WBTC (cost 0.35)
8. Space Allocator : 250B -> block 1 (best fit, 50 bytes left)

Two results are worth highlighting. First, the router chooses the three-hop USDC→ETH→DAI→WBTC path at cost 0.35 over the direct USDC→DAI edge, proving genuine multi-hop optimisation rather than a direct lookup. Second, the allocator places 250 bytes into the 300-byte block (50 left over) instead of the empty 1000-byte block, confirming best-fit behaviour. All modules compile cleanly under -Wall -Wextra with zero warnings.

## 3.11 Conclusion

TokenVault shows that the eight stated platform problems each map cleanly to a classical data structure or algorithm, and that selecting the right one for the access pattern — and justifying it against alternatives — is the core skill the project tests. The Standard Library handles the standard structures idiomatically, while the graph, Dijkstra routing, and best-fit allocator are implemented from scratch to demonstrate algorithmic understanding. The result is a modular, readable, fully executable C++17 system that mirrors how real platforms such as Uniswap and Coinbase organise balances, ordering, routing, and storage.

## 3.12 GitHub Repository Link

<https://github.com/TanishRS/tokenvault>